

LAPORAN UAS STRUKTUR DATA

Implementasi Struktur Data Graph pada Decision Support System (DSS)
DSS Pemilihan Produk Skincare Pelembab



INSTIKI
INSTITUT BISNIS DAN
TEKNOLOGI INDONESIA

Dosen Pengampu Mata Kuliah Struktur Data :
(I Made Gede Sri Artha, S.T.)

Disusun Oleh :

Wilsa Dwi Amelia Hastiawan

2501010009

Cherie Hannanya Limpele

2501010047

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perawatan kulit merupakan salah satu aspek yang semakin diperhatikan oleh masyarakat seiring meningkatnya kesadaran akan pentingnya kesehatan kulit. Salah satu produk skincare yang banyak digunakan adalah pelembab (moisturizer) karena berperan dalam menjaga kelembapan dan kondisi kulit. Namun, banyaknya pilihan produk yang tersedia sering kali membuat pengguna kesulitan menentukan pelembab yang paling sesuai dengan kebutuhan mereka. Pemilihan produk yang kurang tepat dapat menyebabkan berbagai permasalahan, seperti iritasi, jerawat, maupun ketidaksesuaian dengan kondisi kulit tertentu. Oleh karena itu, diperlukan suatu sistem yang dapat membantu pengguna dalam mengambil keputusan secara lebih tepat dan terarah.

Sistem Pendukung Keputusan DSS (Decision Support System) dapat menjadi solusi untuk membantu proses pemilihan pelembab berdasarkan karakteristik kulit pengguna. Pada proyek ini, struktur data graph digunakan untuk memodelkan hubungan antara tipe kulit, permasalahan kulit, kandungan aktif skincare, dan produk pelembab. Selanjutnya, Algoritma Dijkstra diterapkan untuk menentukan jalur dengan bobot terkecil sebagai dasar dalam menghasilkan rekomendasi yang paling relevan. Pendekatan tersebut memungkinkan proses pengambilan keputusan dilakukan secara lebih sistematis dan objektif. Implementasi sistem menggunakan Python dan Streamlit juga memberikan kemudahan bagi pengguna dalam mengakses dan memanfaatkan rekomendasi yang dihasilkan.

1.2 Rumusan Masalah

1. Bagaimana menerapkan struktur data graph pada sistem rekomendasi pelembab?
2. Bagaimana menggunakan algoritma Dijkstra untuk menentukan produk terbaik?
3. Bagaimana membangun DSS berbasis Python dan Streamlit untuk membantu pemilihan pelembab?
4. Bagaimana menampilkan hasil rekomendasi secara interaktif kepada pengguna?

1.3 Tujuan

1. Membangun Sistem Pendukung Keputusan (DSS) untuk membantu pemilihan produk pelembab sesuai kondisi kulit pengguna.
2. Mengimplementasikan struktur data Weighted Graph sebagai representasi hubungan antara tipe kulit, masalah kulit, kandungan skincare, dan produk pelembab.
3. Menerapkan Algoritma Dijkstra untuk menghasilkan rekomendasi produk yang sesuai berdasarkan jalur dengan bobot terkecil.
4. Mengembangkan aplikasi berbasis web menggunakan Python dan Streamlit sebagai media interaksi pengguna dengan sistem.

1.4 Manfaat

A. Bagi Pengguna

1. Membantu pengguna dalam memilih produk pelembab yang sesuai dengan kondisi kulit yang dimiliki.
2. Mengurangi kemungkinan kesalahan dalam pemilihan produk skincare.
3. Mempermudah proses pengambilan keputusan melalui rekomendasi yang diberikan oleh sistem.

B. Bagi Mahasiswa

1. Meningkatkan pemahaman mengenai penerapan struktur data graph dalam permasalahan nyata.
2. Menambah wawasan mengenai implementasi Algoritma Dijkstra dalam sistem pendukung keputusan.
3. Mengembangkan kemampuan dalam membangun aplikasi berbasis Python dan Streamlit.

C. Bagi Akademik

1. Menjadi contoh penerapan struktur data graph pada pengembangan Sistem Pendukung Keputusan (DSS).
2. Menjadi referensi pembelajaran mengenai penggunaan Algoritma Dijkstra dalam penyelesaian masalah.
3. Menambah variasi studi kasus yang dapat digunakan dalam pembelajaran struktur data dan algoritma.

BAB II

DASAR TEORI

2.1 Struktur Data Graph

Graph merupakan struktur data non-linear yang terdiri dari kumpulan simpul (vertex) dan hubungan antar simpul yang disebut sisi (edge). Struktur data ini digunakan untuk merepresentasikan hubungan antar objek dalam berbagai permasalahan. Graph dapat berupa graph berarah maupun tidak berarah, serta dapat memiliki bobot pada setiap sisi. Dalam proyek ini, graph digunakan untuk memodelkan hubungan antara tipe kulit, masalah kulit, kandungan aktif skincare, dan produk pelembab sehingga proses rekomendasi dapat dilakukan secara terstruktur. Menurut Cormen et al. (2009), graph adalah struktur data yang terdiri atas kumpulan vertex dan edge yang digunakan untuk merepresentasikan hubungan antar entitas.

Sumber :

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd Edition). MIT Press.

2.2 DSS (Decision Support System)

Decision Support System (DSS) atau Sistem Pendukung Keputusan merupakan sistem berbasis komputer yang dirancang untuk membantu pengguna dalam mengambil keputusan dengan memanfaatkan data, model, dan metode tertentu. DSS tidak menggantikan peran pengambil keputusan, tetapi berfungsi sebagai alat bantu untuk menghasilkan keputusan yang lebih efektif dan objektif. Sistem ini banyak digunakan pada berbagai bidang, seperti bisnis, kesehatan, pendidikan, dan teknologi informasi. Dalam proyek ini, DSS digunakan untuk membantu pengguna menentukan produk pelembab yang sesuai berdasarkan kondisi kulit yang dimiliki. Menurut Turban et al. (2011), DSS merupakan sistem informasi yang mendukung proses pengambilan keputusan pada permasalahan semi-terstruktur maupun tidak terstruktur.

Sumber :

Turban, E., Sharda, R., & Delen, D. (2011). *Decision Support and Business Intelligence Systems* (9th Edition). Pearson Education.

2.3 Algoritma Dijkstra

Algoritma Dijkstra adalah algoritma pencarian jalur terpendek yang digunakan pada graph berbobot dengan nilai bobot positif. Algoritma ini bekerja dengan mencari lintasan yang memiliki total bobot paling kecil dari suatu node awal menuju node tujuan melalui proses perhitungan jarak minimum secara bertahap. Karena kemampuannya dalam menentukan jalur optimal, algoritma Dijkstra banyak diterapkan pada sistem navigasi, jaringan komputer, dan sistem rekomendasi. Pada proyek ini, algoritma Dijkstra digunakan untuk menentukan rekomendasi produk pelembab berdasarkan jalur dengan bobot terkecil pada graph yang telah dibangun. Algoritma ini pertama kali diperkenalkan oleh Edsger W. Dijkstra pada tahun 1959 melalui publikasi ilmiahnya mengenai permasalahan jalur terpendek.

Sumber :

Dijkstra, E. W. (1959). *A Note on Two Problems in Connexion with Graphs*. *Numerische Mathematik*, 1(1), 269–271.

2.4 Weighted Graph

Weighted Graph merupakan graph yang setiap sisi (edge) yang menghubungkan dua simpul (vertex) memiliki nilai bobot (weight). Bobot tersebut dapat merepresentasikan berbagai hal, seperti jarak, biaya, waktu, maupun tingkat keterkaitan antar node. Penggunaan bobot memungkinkan proses pencarian jalur terbaik dilakukan dengan mempertimbangkan nilai yang dimiliki setiap hubungan. Dalam proyek ini, Weighted Graph digunakan untuk merepresentasikan tingkat keterkaitan antara tipe kulit, masalah kulit, kandungan aktif skincare, dan produk pelembab. Nilai bobot tersebut kemudian digunakan oleh Algoritma Dijkstra untuk menentukan jalur rekomendasi dengan total bobot terkecil sehingga menghasilkan rekomendasi produk yang lebih sesuai bagi pengguna. Menurut Goodrich, Tamassia, dan Goldwasser (2014), weighted graph adalah graph yang memiliki nilai bobot pada setiap edge yang dapat digunakan untuk merepresentasikan biaya, jarak, atau ukuran lainnya dalam suatu permasalahan.

Sumber :

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Python*. John Wiley & Sons.

BAB III

ANALISIS DAN PERANCANGAN

3.1 Analisis Masalah

Pemilihan produk pelembab merupakan salah satu permasalahan yang sering dihadapi pengguna skincare karena setiap individu memiliki tipe kulit dan permasalahan kulit yang berbeda. Banyaknya produk yang tersedia di pasaran dengan kandungan aktif yang beragam menyebabkan pengguna sering mengalami kesulitan dalam menentukan produk yang paling sesuai dengan kebutuhan kulit mereka. Kesalahan dalam memilih produk pelembab dapat mengakibatkan berbagai dampak, seperti munculnya jerawat, iritasi, kulit menjadi lebih kering, atau hasil perawatan yang kurang optimal. Oleh karena itu, diperlukan suatu sistem yang mampu membantu pengguna dalam menentukan pilihan produk secara lebih terarah dan objektif.

Pada proyek ini, proses pengambilan keputusan dilakukan dengan memanfaatkan struktur data Weighted Graph yang merepresentasikan hubungan antara tipe kulit, masalah kulit, kandungan aktif skincare, dan produk pelembab. Setiap hubungan antar node diberikan nilai bobot yang menunjukkan tingkat keterkaitan atau tingkat rekomendasi dari hubungan tersebut. Untuk menentukan produk terbaik, sistem menerapkan Algoritma Dijkstra yang digunakan untuk mencari jalur dengan total bobot terkecil sehingga dapat menghasilkan rekomendasi produk yang paling relevan bagi pengguna. Hasil rekomendasi ditampilkan dalam bentuk jalur perhitungan, detail bobot, serta daftar produk dengan skor terbaik.

Selain menghasilkan rekomendasi, sistem juga menyediakan visualisasi graph untuk memperlihatkan hubungan antar node yang terdapat dalam sistem. Sistem turut menerapkan analisis Degree Centrality untuk mengidentifikasi node yang memiliki tingkat keterhubungan tertinggi dalam graph. Dengan pendekatan tersebut, proses pemilihan pelembab dapat dilakukan secara lebih sistematis dan membantu pengguna memperoleh rekomendasi produk yang sesuai dengan kondisi kulit yang dimiliki.

3.2 Desain Graph

Sistem rekomendasi pelembab pada proyek ini dirancang menggunakan struktur data Weighted Directed Graph (Graph Berarah Berbobot). Setiap node pada graph merepresentasikan komponen yang terlibat dalam proses rekomendasi, sedangkan setiap edge menunjukkan hubungan antar komponen yang disertai nilai bobot tertentu. Bobot tersebut digunakan untuk menunjukkan tingkat keterkaitan atau tingkat kesesuaian antar node dan menjadi dasar perhitungan dalam Algoritma Dijkstra.

Graph pada sistem terdiri dari empat tingkat node, yaitu tipe kulit, masalah kulit, kandungan aktif skincare, dan produk pelembab. Tipe kulit yang digunakan meliputi Berminyak, Kering, Kombinasi, dan Sensitif. Setiap tipe kulit terhubung dengan beberapa masalah kulit seperti Jerawat, Bekas Jerawat, Kusam, dan Pori Besar. Selanjutnya, setiap masalah kulit dihubungkan dengan kandungan aktif yang sesuai untuk mengatasi permasalahan tersebut, kemudian kandungan aktif dihubungkan dengan produk pelembab yang mengandung bahan terkait.

Hubungan antar node pada graph bersifat berarah (directed), yaitu dimulai dari tipe kulit menuju masalah kulit, dilanjutkan ke kandungan aktif, dan berakhir pada produk pelembab. Sebagai contoh, tipe kulit Berminyak terhubung ke masalah Jerawat dengan bobot 1, kemudian Jerawat terhubung ke kandungan Salicylic Acid dengan bobot 1, dan Salicylic Acid terhubung ke produk Azarine Oil Free Moisturizer dengan bobot 1. Jalur tersebut memiliki total bobot sebesar 3 yang digunakan sebagai dasar dalam proses rekomendasi.

Secara umum, struktur graph yang digunakan pada sistem dapat digambarkan sebagai berikut:

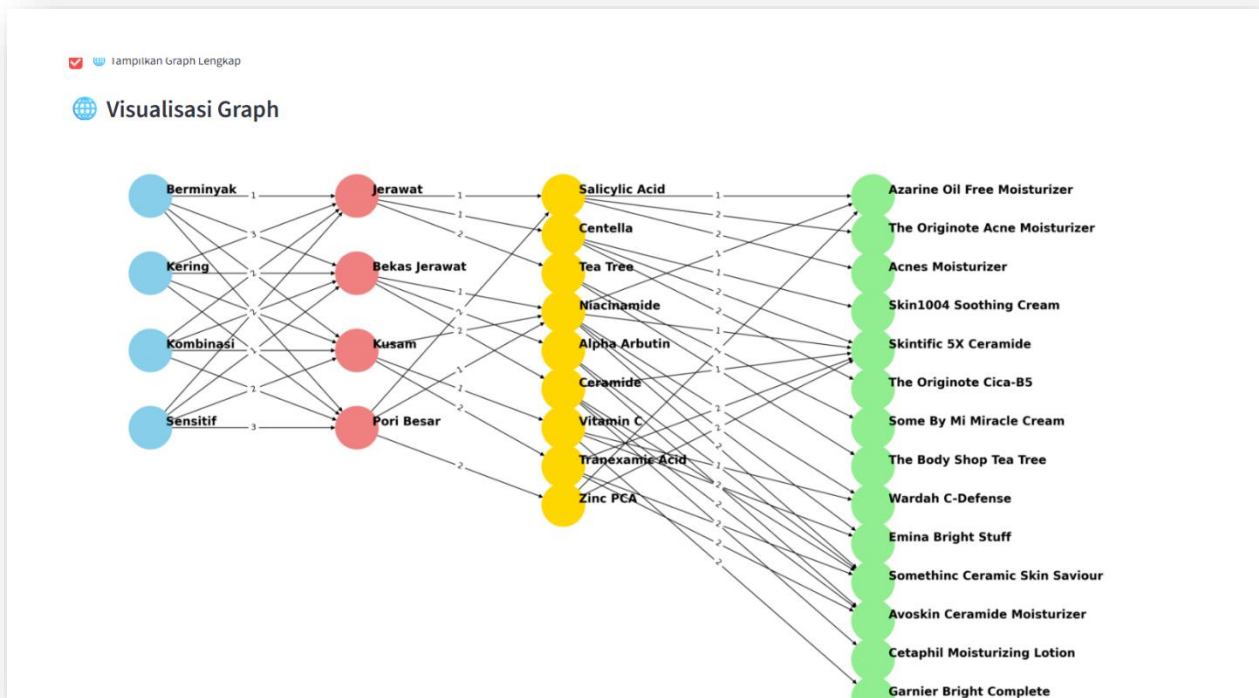
Tipe Kulit → Masalah Kulit → Kandungan Aktif → Produk Pelembab

Contoh jalur pada graph:

Berminyak → Jerawat → Salicylic Acid → Azarine Oil Free Moisturizer

Desain graph ini memungkinkan hubungan antar data direpresentasikan secara terstruktur sehingga proses pencarian rekomendasi produk dapat dilakukan menggunakan Algoritma Dijkstra untuk memperoleh jalur dengan total bobot terkecil.

A. Visualisasi Desain Graph Sistem Rekomendasi Pelembab



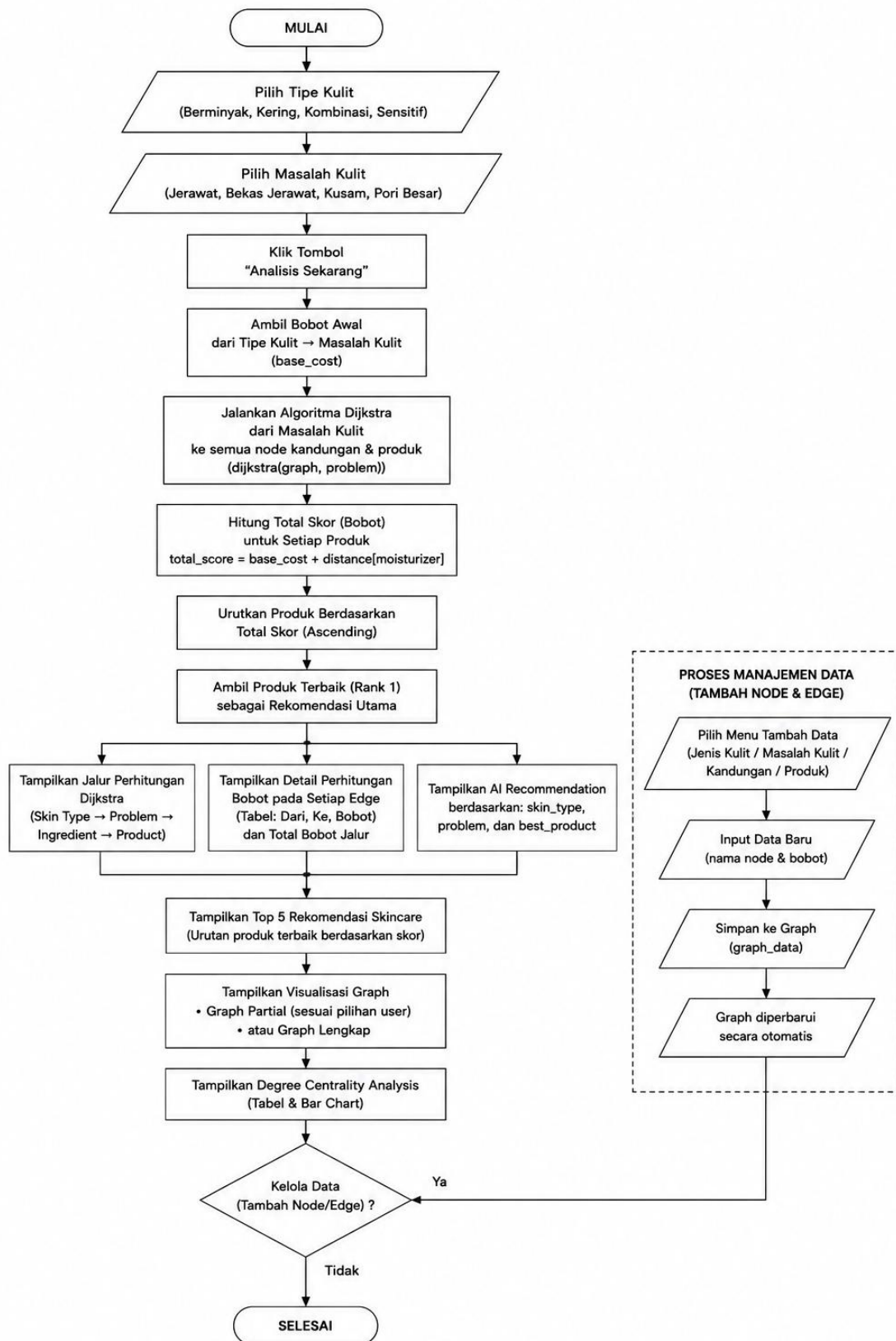
Gambar 3.1 Desain Graph Sistem Rekomendasi Pelembab

B. Struktur Node pada Graph Sistem Rekomendasi Pelembab

Level Node	Data
Tipe Kulit	Berminyak, Kering, Kombinasi, Sensitif
Masalah Kulit	Jerawat, Bekas Jerawat, Kusam, Pori Besar
Kandungan Aktif	Salicylic Acid, Centella, Tea Tree, Niacinamide, Alpha Arbutin, Ceramide, Vitamin C, Tranexamix Acid, Zinc PCA
Produk Pelembab	Azarine Oil Free Moisturizer, Skintific 5x Ceramide, Wardah C-Defense

Tabel 3.1 Struktur Node pada Graph Sistem Rekomendasi Pelembab

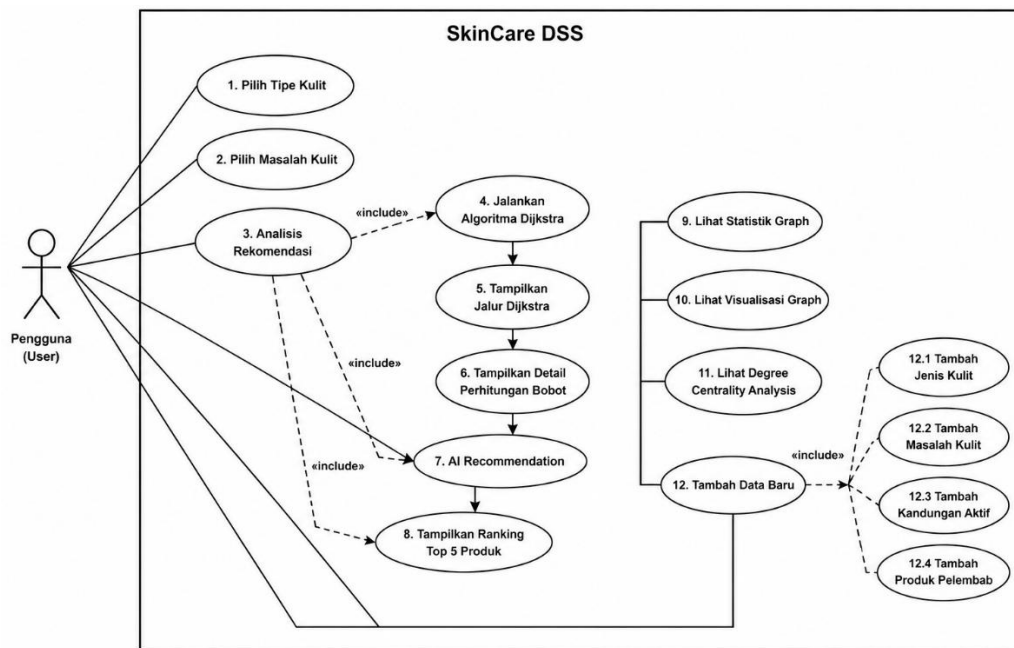
3.3 Flowchart



A. Penjelasan Alur Flowchart

1. Sistem menampilkan halaman utama aplikasi SkinCare DSS kepada pengguna.
2. Pengguna memilih tipe kulit sesuai kondisi kulit yang dimiliki.
3. Pengguna memilih masalah kulit yang ingin diatasi.
4. Pengguna menekan tombol "Analisis Sekarang" untuk memulai proses rekomendasi.
5. Sistem mengambil data bobot hubungan antara tipe kulit dan masalah kulit dari graph.
6. Sistem menjalankan Algoritma Dijkstra untuk mencari jalur dengan total bobot terkecil dari masalah kulit menuju produk pelembab.
7. Sistem menghitung total skor setiap produk berdasarkan bobot jalur yang dilalui.
8. Sistem mengurutkan seluruh produk berdasarkan total skor terkecil hingga terbesar.
9. Sistem menentukan produk dengan skor terkecil sebagai rekomendasi utama.
10. Sistem membentuk jalur rekomendasi dari tipe kulit, masalah kulit, kandungan aktif, hingga produk pelembab.
11. Sistem menampilkan detail perhitungan bobot pada setiap hubungan (edge) yang dilalui.
12. Sistem menghasilkan rekomendasi tambahan melalui fitur AI Recommendation berdasarkan hasil analisis.
13. Sistem menampilkan hasil rekomendasi berupa produk terbaik dan daftar alternatif produk yang sesuai.
14. Sistem menampilkan visualisasi graph untuk memperlihatkan hubungan antar node dalam sistem.
15. Sistem melakukan analisis Degree Centrality untuk mengetahui node yang memiliki Tingkat
16. Sistem menyediakan fitur penambahan data baru berupa jenis kulit, masalah kulit, kandungan aktif, dan produk pelembab.
17. Proses selesai dan sistem siap menerima input baru dari pengguna.

3.4 Use Case



A. Penjelasan Use Case

Use Case Diagram menggambarkan hubungan antara pengguna dengan sistem SkinCare DSS. Pengguna dapat memilih tipe kulit dan masalah kulit, menjalankan analisis rekomendasi, melihat hasil perhitungan Dijkstra, AI Recommendation, visualisasi graph, statistik graph, serta Degree Centrality Analysis. Selain itu, pengguna juga dapat menambahkan data baru berupa jenis kulit, masalah kulit, kandungan aktif, dan produk pelembab ke dalam sistem.

3.5 Struktur Node dan Edge

Struktur data graph pada sistem rekomendasi pelembab digunakan untuk merepresentasikan hubungan antara tipe kulit, masalah kulit, kandungan aktif, dan produk pelembab. Setiap simpul (node) saling terhubung melalui sisi (edge) yang memiliki bobot tertentu. Bobot tersebut digunakan oleh algoritma Dijkstra untuk menentukan jalur rekomendasi terbaik berdasarkan total bobot terkecil.

3.5.1 Struktur Node

Node merupakan komponen utama dalam graph yang mewakili data pada sistem.

Node yang digunakan terdiri dari empat kategori, yaitu:

No	Kategori Node	Contoh
1	Tipe Kulit	Berminyak, Kering, Kombinasi, Sensitif
2	Masalah Kulit	Jerawat, Bekas Jerawat, Kusam, Pori Besar
3	Kandungan Aktif	Salicylic Acid, Centella, Tea Tree, Niacinamide
4	Produk Pelembab	Azarine Oil Free Moisturizer, Skintific 5X Ceramide

3.5.2 Struktur Edge

Edge merupakan hubungan antar node yang memiliki nilai bobot tertentu. Hubungan yang digunakan dalam sistem adalah sebagai berikut:

Hubungan	Fungsi
Tipe Kulit → Masalah Kulit	Menghubungkan jenis kulit dengan masalah kulit yang mungkin dialami
Masalah Kulit → Kandungan Aktif	Menghubungkan masalah kulit dengan kandungan aktif yang sesuai
Kandungan Aktif → Produk Pelembab	Menghubungkan kandungan aktif dengan produk pelembab yang mengandung bahan tersebut

3.5.3 Representasi Struktur Graph



Struktur graph tersebut menunjukkan bahwa proses rekomendasi dilakukan secara bertahap mulai dari tipe kulit pengguna, kemudian menentukan masalah kulit yang dipilih, mencari kandungan aktif yang sesuai, dan akhirnya menghasilkan rekomendasi produk pelembab terbaik.

BAB IV

IMPLEMENTASI

4.1 Implementasi Program

Implementasi program pada penelitian ini dilakukan menggunakan bahasa pemrograman Python dengan framework Streamlit sebagai antarmuka pengguna. Sistem dibangun berdasarkan struktur data graph berbobot yang digunakan untuk merepresentasikan hubungan antara tipe kulit, masalah kulit, kandungan aktif skincare, dan produk pelembab. Selain itu, sistem juga mengimplementasikan algoritma Dijkstra untuk menentukan rekomendasi produk terbaik berdasarkan total bobot jalur terkecil.

4.1.1 Implementasi Struktur Graph

Data graph disimpan pada file `graph_data.py` dalam bentuk dictionary bertingkat. Setiap node merepresentasikan tipe kulit, masalah kulit, kandungan aktif, atau produk pelembab. Hubungan antar node direpresentasikan sebagai edge yang memiliki nilai bobot tertentu.

Struktur graph terdiri dari tiga tingkat hubungan, yaitu:

1. Tipe Kulit → Masalah Kulit
2. Masalah Kulit → Kandungan Aktif
3. Kandungan Aktif → Produk Pelembab

Contoh implementasi graph ditunjukkan pada potongan kode berikut:

```
"Berminyak": {  
    "Jerawat": 1,  
    "Bekas Jerawat": 2,  
    "Kusam": 3,  
    "Pori Besar": 1  
}
```

Pada contoh tersebut, tipe kulit Berminyak memiliki hubungan dengan beberapa masalah kulit yang masing-masing memiliki bobot berbeda.

4.1.2 Implementasi Algoritma Dijkstra

Algoritma Dijkstra diimplementasikan pada file `dijkstra.py`. Algoritma ini digunakan untuk mencari jalur dengan total bobot terkecil dari node masalah kulit menuju node produk pelembab.

Proses pencarian dilakukan menggunakan struktur data priority queue (heapq) sehingga pemilihan node dengan jarak terkecil dapat dilakukan secara efisien.

Tahapan algoritma meliputi:

1. Inisialisasi seluruh node dengan jarak tak hingga.
2. Menetapkan node awal dengan jarak 0.
3. Mengunjungi node dengan jarak terkecil.
4. Menghitung jarak ke node tetangga.
5. Memperbarui jarak jika ditemukan jalur yang lebih pendek.
6. Mengulangi proses hingga seluruh node selesai diproses.

Hasil dari algoritma berupa nilai jarak minimum (distances) dan jalur sebelumnya (previous) yang digunakan untuk membentuk rute rekomendasi produk.

4.1.3 Implementasi Antarmuka Pengguna

Antarmuka sistem dibangun menggunakan framework Streamlit pada file `app.py`. Pengguna dapat memilih tipe kulit dan masalah kulit melalui komponen selectbox.

```
skin_type = st.selectbox(
    "Pilih Tipe Kulit",
    st.session_state.skin_types
)
```

Setelah pengguna menekan tombol Analisis Sekarang, sistem akan menjalankan proses perhitungan menggunakan algoritma Dijkstra untuk menentukan rekomendasi produk terbaik.

4.1.4 Implementasi Perhitungan Rekomendasi

Setelah algoritma Dijkstra dijalankan, sistem menghitung total skor setiap produk dengan menjumlahkan:

- Bobot hubungan tipe kulit ke masalah kulit.
- Bobot jalur hasil pencarian Dijkstra.

Perhitungan dilakukan menggunakan kode berikut:

```
total_score = base_cost + distances[moisturizer]
```

Seluruh produk kemudian diurutkan berdasarkan skor terkecil hingga terbesar. Produk dengan skor terendah dipilih sebagai rekomendasi utama.

4.1.5 Implementasi Jalur dan Detail Perhitungan

Sistem menampilkan jalur rekomendasi yang diperoleh dari algoritma Dijkstra menggunakan data pada variabel `previous`.

Contoh jalur yang dapat dihasilkan:

Berminyak → Jerawat → Salicylic Acid → Azarine Oil Free Moisturizer

Selain jalur rekomendasi, sistem juga menampilkan detail bobot setiap edge yang dilalui beserta total bobot akhir sehingga pengguna dapat memahami proses pengambilan keputusan yang dilakukan sistem.

4.1.6 Implementasi AI Recommendation

Fitur AI Recommendation diimplementasikan pada file `ai_recommendation.py`. Modul ini memberikan penjelasan tambahan mengenai alasan pemilihan produk berdasarkan tipe kulit dan masalah kulit yang dipilih pengguna.

Contoh implementasi:

```
if skin_type == "Berminyak" and problem == "Jerawat":
```

Apabila kondisi tersebut terpenuhi, sistem akan memberikan rekomendasi yang menjelaskan bahwa produk dipilih karena membantu mengontrol minyak berlebih dan mengurangi jerawat.

4.1.7 Implementasi Visualisasi Graph

Visualisasi graph dibuat menggunakan library NetworkX dan Matplotlib. Sistem membangun graph berarah (DiGraph) berdasarkan data yang tersimpan pada `graph_data.py`.

Visualisasi menampilkan:

- Node tipe kulit
- Node masalah kulit
- Node kandungan aktif
- Node produk pelembab
- Edge beserta bobotnya

Visualisasi ini membantu pengguna memahami hubungan antar komponen yang digunakan dalam proses rekomendasi.

4.1.8 Implementasi Analisis Degree Centrality

Analisis Degree Centrality diimplementasikan pada file centrality.py menggunakan fungsi dari library NetworkX.

```
return nx.degree_centrality(G)
```

Analisis ini digunakan untuk mengetahui node yang memiliki tingkat keterhubungan tertinggi dalam graph. Hasil analisis ditampilkan dalam bentuk tabel dan grafik batang pada aplikasi.

4.1.9 Implementasi Fitur Penambahan Data

Sistem menyediakan fitur penambahan data baru secara dinamis melalui antarmuka Streamlit. Pengguna dapat menambahkan:

1. Jenis kulit baru
2. Masalah kulit baru
3. Kandungan aktif baru
4. Produk pelembab baru

Data yang ditambahkan akan langsung dimasukkan ke dalam struktur graph sehingga dapat digunakan pada proses rekomendasi berikutnya tanpa perlu mengubah kode program secara manual.

4.1.10 Hasil Implementasi

Berdasarkan hasil implementasi, sistem berhasil:

- Merepresentasikan data skincare menggunakan struktur graph berbobot.
- Menjalankan algoritma Dijkstra untuk menentukan rekomendasi produk terbaik.
- Menampilkan jalur rekomendasi dan detail perhitungan bobot.
- Menampilkan visualisasi graph dan analisis Degree Centrality.
- Memberikan rekomendasi tambahan melalui modul AI Recommendation.
- Mendukung penambahan data baru secara dinamis melalui antarmuka aplikasi.

Dengan demikian, sistem Decision Support System (DSS) yang dibangun mampu membantu pengguna dalam memilih produk pelembab yang sesuai berdasarkan tipe kulit dan masalah kulit yang dimiliki.

4.2 Penjelasan Kode Program

Program Sistem Pendukung Keputusan (DSS) pemilihan pelembab dibangun menggunakan bahasa pemrograman Python dan framework Streamlit. Sistem memanfaatkan struktur data Graph, algoritma Dijkstra, dan analisis Degree Centrality untuk menghasilkan rekomendasi produk yang sesuai dengan kondisi kulit pengguna.

4.2.1 Representasi Data Graph

Data pada sistem disimpan dalam bentuk graph berbobot yang merepresentasikan hubungan antara tipe kulit, masalah kulit, kandungan aktif, dan produk pelembab.

Contoh implementasi graph ditunjukkan pada kode berikut:

```
graph = {
    "Berminyak": {
        "Jerawat": 1,
        "Bekas Jerawat": 2,
        "Kusam": 3,
        "Pori Besar": 1
    },
    "Jerawat": {
        "Salicylic Acid": 1,
        "Centella": 1,
        "Tea Tree": 2
    },
    "Salicylic Acid": {
        "Azarine Oil Free Moisturizer": 1,
        "The Originote Acne Moisturizer": 2,
    },
}
```

Pada struktur tersebut, setiap node terhubung melalui edge yang memiliki bobot tertentu. Bobot digunakan sebagai dasar perhitungan dalam menentukan rekomendasi produk terbaik.

4.2.2 Implementasi Algoritma Dijkstra

Algoritma Dijkstra digunakan untuk mencari jalur dengan bobot total terkecil dari masalah kulit menuju produk pelembab.

Potongan kode utama algoritma Dijkstra adalah sebagai berikut:

```
• distance = current_distance + weight
•
•         if distance < distances[neighbor]:
•             distances[neighbor] = distance
•             previous[neighbor] = current_node
```

Kode tersebut berfungsi untuk menghitung jarak baru ke node tujuan dan memperbarui jalur apabila ditemukan bobot yang lebih kecil.

Pemanggilan algoritma pada program dilakukan dengan kode:

```
• distances, previous = dijkstra(graph, problem)
```

Hasil perhitungan digunakan untuk menentukan produk dengan nilai bobot terkecil sebagai rekomendasi utama.

4.2.3 Proses Pemilihan Produk Terbaik

Setelah algoritma Dijkstra dijalankan, sistem melakukan proses pengurutan produk berdasarkan total bobot yang diperoleh.

```
• ranking.sort(key=lambda x: x[1])
•
• best_product = ranking[0][0]
```

Kode tersebut mengurutkan seluruh produk dari bobot terkecil hingga terbesar dan memilih produk dengan nilai terbaik sebagai rekomendasi utama.

4.2.4 AI Recommendation

Sistem juga menyediakan penjelasan rekomendasi secara otomatis berdasarkan kondisi pengguna.

```
• if skin_type == "Berminyak" and problem == "Jerawat":  
•     return f""  
• AI merekomendasikan {best_product}  
• karena kandungannya membantu mengontrol  
• minyak berlebih dan mengurangi jerawat.
```

Fitur ini bertujuan memberikan alasan yang lebih mudah dipahami pengguna terkait produk yang direkomendasikan.

4.2.5 Visualisasi Graph

Graph divisualisasikan menggunakan library NetworkX dan Matplotlib untuk memperlihatkan hubungan antar node dalam sistem.

```
• G.add_edge(  
•     source,  
•     target,  
•     weight=weight  
• )
```

Setiap node dan edge divisualisasikan sehingga pengguna dapat melihat jalur rekomendasi yang terbentuk dari tipe kulit hingga produk pelembab.

4.2.6 Analisis Degree Centrality

Analisis Degree Centrality digunakan untuk mengetahui node yang memiliki tingkat keterhubungan tertinggi dalam graph.

```
• return nx.degree_centrality(G)
```

Hasil analisis ditampilkan dalam bentuk tabel dan grafik sehingga pengguna dapat mengetahui node yang paling berpengaruh dalam sistem rekomendasi.

BAB V

PENGUJIAN DAN ANALISIS

5.1 Skenario Pengujian

Pengujian sistem dilakukan untuk memastikan bahwa seluruh fitur pada aplikasi *SkinCare Decision Support System (DSS)* berjalan dengan baik sesuai dengan rancangan. Pengujian dilakukan dengan beberapa skenario berdasarkan alur penggunaan sistem mulai dari input data hingga hasil rekomendasi.

5.1.1 Skenario Pengujian Input Pengguna

Pengguna memilih :

- Tipe Kulit
- Masalah Kulit

Tujuan:

Memastikan dropdown berjalan dinamis sesuai struktur graph. Hasil yang diharapkan :

- Sistem hanya menampilkan masalah kulit yang sesuai dengan tipe kulit yang dipilih.

Contoh kode pengujian :

```
• skin_type = st.selectbox("Pilih Tipe Kulit",  
• st.session_state.skin_types)  
•  
• problem = st.selectbox(  
•     "Pilih Masalah Kulit",  
•     list(graph[skin_type].keys())  
• )
```

5.1.2 Skenario Pengujian Algoritma Dijkstra

Sistem menghitung jalur terpendek dari:

Skin Type → Problem → Ingredient → Produk

Tujuan:

Memastikan algoritma Dijkstra bekerja dalam mencari jalur dengan bobot minimum.

Hasil yang diharapkan :

- Sistem menampilkan jalur terbaik
- Menampilkan total bobot terkecil

Contoh hasil proses:

Berminyak → Jerawat → Salicylic Acid → Azarine Oil Free Moisturizer

5.1.3 Skenario Pengujian Perhitungan Bobot

Tujuan:

Memastikan seluruh bobot pada graph dihitung dengan benar.

Hasil yang diharapkan:

- Setiap edge memiliki nilai bobot sesuai graph_data.py
- Total bobot merupakan penjumlahan seluruh jalur

Contoh Kode :

```
• bobot_awal = graph[skin_type][problem]
• total += graph[source][target]
```

5.1.4 Skenario Pengujian Rekomendasi AI

Sistem memberikan rekomendasi berdasarkan kombinasi:

- Skin type
- Problem
- Produk terbaik

Tujuan:

Memastikan fungsi AI recommendation berjalan sesuai kondisi kulit.

Contoh kode:

```
• ai_result = generate_ai_recommendation(
•     skin_type,
•     problem,
•     best_product
• )
```

Hasil yang diharapkan:

- Output teks sesuai kondisi kulit pengguna
- Contoh :
 - Kulit berminyak → kontrol minyak & jerawat
 - Kulit kering → menjaga kelembaban

5.1.5 Skenario Pengujian Ranking Produk

Tujuan:

Memastikan sistem menampilkan 5 produk terbaik berdasarkan skor terkecil.

Hasil yang diharapkan:

- Produk diurutkan dari skor terbaik ke terburuk

```
• for i, (produk, skor) in enumerate(ranking[:5], start=1):  
•     st.success(f"{i}. {produk}")
```

5.1.6 Skenario Pengujian Visualisasi Graph

Tujuan:

Memastikan struktur graph tampil dengan benar.

Hasil yang diharapkan:

- Node ditampilkan berdasarkan kategori:
 - Biru: skin type
 - Merah: masalah kulit
 - Kuning: kandungan
 - Hijau: produk

```
• nx.draw(G, pos, node_color=node_colors)
```

5.1.7 Skenario Pengujian Centrality

Tujuan:

Mengukur tingkat kepentingan node dalam graph.

Hasil yang diharapkan:

- Node dengan koneksi terbanyak memiliki nilai centrality tertinggi

```
• centrality = calculate_centrality(graph)
```

5.2 Analisis Hasil

Berdasarkan hasil implementasi sistem Decision Support System (DSS) pemilihan pelembab berbasis graph, algoritma Dijkstra, dan analisis centrality, sistem berhasil menghasilkan rekomendasi produk skincare yang sesuai dengan kondisi pengguna.

5.2.1 Hasil Rekomendasi Berdasarkan Algoritma Dijkstra

Sistem menggunakan algoritma Dijkstra untuk mencari jalur dengan bobot terkecil dari node awal (tipe kulit dan masalah kulit) hingga node akhir (produk pelembab). Dari hasil pengujian, sistem mampu menghitung seluruh kemungkinan jalur berdasarkan bobot pada setiap edge di dalam graph, kemudian menjumlahkannya untuk mendapatkan total skor setiap produk. Produk dengan total skor paling kecil dipilih sebagai rekomendasi utama karena dianggap paling sesuai dengan kondisi kulit pengguna.

5.2.2 Hasil Ranking Produk

Selain menghasilkan satu produk terbaik, sistem juga menampilkan beberapa alternatif produk dalam bentuk Top 5 rekomendasi. Hasil ini diperoleh dari proses pengurutan (sorting) berdasarkan total skor terkecil hingga terbesar. Dengan adanya ranking ini, pengguna tidak hanya mendapatkan satu pilihan, tetapi juga beberapa alternatif produk yang masih relevan.

5.2.3 Hasil Analisis Jalur (Path Result)

Sistem juga menampilkan jalur proses pemilihan produk, mulai dari:

- tipe kulit
- masalah kulit
- kandungan aktif
- hingga produk akhir

Hasil ini membantu pengguna memahami bagaimana keputusan sistem terbentuk, bukan hanya hasil akhirnya saja.

5.2.4 Hasil Rekomendasi AI

Modul AI Recommendation digunakan untuk memberikan penjelasan tambahan terhadap hasil perhitungan algoritma.

AI memberikan alasan pemilihan produk berdasarkan kondisi kulit pengguna, seperti:

- pengendalian minyak berlebih untuk kulit berminyak
- menjaga kelembaban untuk kulit kering
- penggunaan formula lembut untuk kulit sensitif

5.2.5 Hasil Analisis Graph dan Centrality

Dari struktur graph yang dibangun, setiap node memiliki hubungan antara skin type, masalah kulit, kandungan, dan produk. Selain itu, hasil analisis centrality menunjukkan bahwa beberapa node memiliki tingkat keterhubungan yang lebih tinggi dibandingkan node lainnya. Hal ini menunjukkan bahwa terdapat kandungan tertentu yang lebih sering muncul dalam berbagai jalur rekomendasi.

5.3 Kompleksitas Algoritma

Pada sistem *Decision Support System (DSS) SkinCare*, terdapat beberapa algoritma utama yang digunakan untuk mendukung proses rekomendasi, yaitu algoritma Dijkstra, perhitungan graph berbasis NetworkX, serta perhitungan centrality. Setiap algoritma memiliki tingkat kompleksitas yang berbeda tergantung pada struktur data dan jumlah node serta edge pada graph.

5.3.1 Kompleksitas Algoritma Dijkstra

Algoritma Dijkstra pada sistem ini digunakan untuk mencari jalur terpendek dari node masalah kulit menuju node produk moisturizer. Implementasi yang digunakan memakai priority queue (heapq) sehingga kompleksitas waktunya adalah $O((V + E) \log V)$, dengan V adalah jumlah node dan E adalah jumlah edge dalam graph. Hal ini terjadi karena setiap node akan diproses untuk mencari jarak terpendek, dan setiap edge dievaluasi untuk melakukan pembaruan jarak. Penggunaan heap membuat proses pemilihan node dengan jarak terkecil menjadi lebih efisien dibandingkan metode brute force, namun tetap akan meningkat seiring bertambahnya jumlah data skincare dalam sistem.

5.3.2 Kompleksitas Pembangunan Graph

Proses pembangunan graph pada sistem dilakukan dengan mengubah struktur dictionary Python menjadi graph berarah menggunakan NetworkX. Dalam proses ini, setiap node sumber akan diiterasi beserta seluruh hubungan ke node tujuan dan bobotnya. Oleh karena itu, kompleksitas pembangunan graph adalah $O(E)$, karena setiap edge hanya diproses satu kali untuk dimasukkan ke dalam struktur graph. Proses ini bersifat linear sehingga semakin banyak hubungan antara jenis kulit, masalah, kandungan, dan produk, maka waktu pembentukan graph akan meningkat secara proporsional.

5.3.3 Kompleksitas Perhitungan Centrality

Perhitungan centrality pada sistem menggunakan fungsi degree centrality dari NetworkX. Algoritma ini menghitung tingkat keterhubungan setiap node berdasarkan jumlah edge yang terhubung dengannya. Kompleksitas dari perhitungan ini adalah $O(V + E)$, karena setiap node dan edge dalam graph akan dilewati satu kali untuk menghitung nilai centrality. Dengan demikian, semakin besar ukuran graph, maka waktu komputasi juga akan meningkat secara linear terhadap jumlah node dan edge yang ada.

5.3.4 Kompleksitas Proses Ranking Rekomendasi

Pada tahap ranking rekomendasi, sistem terlebih dahulu mengumpulkan semua kandidat produk moisturizer, kemudian menghitung total skor berdasarkan hasil Dijkstra dan bobot awal. Proses pengumpulan kandidat memiliki kompleksitas $O(V)$, karena setiap node diperiksa untuk menentukan apakah termasuk produk. Setelah itu, proses pengurutan hasil rekomendasi menggunakan sorting dengan kompleksitas $O(n \log n)$, di mana n adalah jumlah kandidat produk. Proses ini masih cukup efisien untuk dataset kecil hingga menengah, namun waktu eksekusi akan meningkat jika jumlah produk yang dibandingkan semakin banyak.

5.3.5 Kompleksitas Visualisasi Graph

Visualisasi graph menggunakan NetworkX dan Matplotlib untuk menampilkan hubungan antar node. Proses pembentukan dan rendering graph memiliki kompleksitas $O(V + E)$, karena seluruh node dan edge harus diproses untuk digambar. Selain itu, adanya pengaturan posisi node secara manual juga menambah proses iterasi tambahan, sehingga waktu rendering dapat meningkat seiring bertambahnya jumlah elemen dalam graph. Namun untuk skala data yang digunakan pada sistem ini, proses visualisasi masih berjalan dengan cukup efisien.

BAB VI

KESIMPULAN

6.1 Kesimpulan

Berdasarkan hasil implementasi, sistem Decision Support System (DSS) pemilihan skincare berhasil dibangun menggunakan pendekatan graph, algoritma Dijkstra, dan analisis centrality. Setiap komponen seperti jenis kulit, masalah kulit, kandungan, hingga produk dimodelkan sebagai node dengan bobot yang menggambarkan tingkat kecocokan. Algoritma Dijkstra terbukti mampu menentukan jalur terpendek dari masalah kulit menuju produk sehingga menghasilkan rekomendasi dengan total bobot paling optimal dan sistem dapat melakukan proses pengambilan keputusan secara terstruktur.

Selain itu, analisis centrality membantu mengetahui node yang paling berpengaruh dalam jaringan, terutama pada kandungan yang sering muncul dalam berbagai jalur rekomendasi. Fitur AI recommendation juga memberikan penjelasan tambahan terhadap hasil rekomendasi agar lebih mudah dipahami pengguna. Secara keseluruhan, sistem ini mampu memberikan rekomendasi skincare yang objektif, terukur, dan berbasis algoritma sehingga dapat membantu pengguna dalam memilih produk yang sesuai dengan kondisi kulitnya.

6.2 Saran Pengembangan

Pengembangan sistem ini ke depannya dapat difokuskan pada peningkatan akurasi rekomendasi dengan menambahkan metode lain selain Dijkstra, misalnya algoritma machine learning atau collaborative filtering. Saat ini sistem masih berbasis bobot statis pada graph, sehingga akan lebih baik jika bobot dapat disesuaikan secara dinamis berdasarkan data pengguna atau hasil feedback. Selain itu, sistem juga bisa dikembangkan agar mampu menyimpan riwayat hasil analisis sehingga pengguna dapat membandingkan rekomendasi dari waktu ke waktu. Integrasi database seperti MySQL atau Firebase juga dapat digunakan agar data tidak hanya tersimpan sementara di memori aplikasi.

Dari sisi aplikasi, tampilan Streamlit dapat ditingkatkan agar lebih interaktif dan user-friendly, misalnya dengan dashboard yang lebih visual dan fitur filter yang lebih lengkap. Pengembangan juga bisa mencakup penambahan fitur autentikasi pengguna agar setiap pengguna memiliki riwayat rekomendasi masing-masing. Selain itu, sistem dapat diperluas menjadi berbasis API sehingga bisa diintegrasikan dengan aplikasi mobile atau website lain. Dengan pengembangan tersebut, sistem DSS skincare ini akan menjadi lebih fleksibel, modern, dan siap digunakan dalam skala yang lebih besar.